How to Set Up Kafka with Multiple Partitions and Consumers: A Step-by-Step Guide

systemdesignschool.io/fundamentals/kafka-tutorial

Kafka Exercise

In this exercise, we are going to demonstrate how Kafka works with multiple partitions and consumers. We will

- create a new topic, with 3 partitions
- have a producer sending ordered integer values as messages to the topic
- have 3 consumers each assigned to a partition in the topic
- observe how messages are consumed by the consumers

Installing and setting up Kafka is more complex than Redis. You need to install JDK, Kafka, Python and set a few other things up.

For quick practice, we provide a Docker environment:

```
docker run -dit --name kafka-tutorial-practice systemdesignschool/kafka-tutorial
tail -f /dev/null
```

```
~ %>docker run -dit --name kafka-tutorial-practice systemdesign
school/kafka-tutorial tail -f /dev/null
Unable to find image 'systemdesignschool/kafka-tutorial:latest' locally
latest: Pulling from systemdesignschool/kafka-tutorial
3610fc6cdcbd: Already exists
440ff9e33d74: Already exists
0dc0d0d23e1f: Already exists
e9246f522e03: Already exists
65ff5556e28a: Already exists
2756c41a3586: Already exists
038e9308aaee: Already exists
0337bd56182e: Already exists
55016d519ba7: Already exists
3aab5d410804: Already exists
a4d1be64c31f: Already exists
393dc9c7ee4a: Already exists
06b7a9368c31: Already exists
Digest: sha256:bb118e827dfb6e22a3f2f5f0caced97d246fb9067600ad870eaa1b6f3116799d
Status: Downloaded newer image for systemdesignschool/kafka-tutorial:latest
ca7cc92d6d51b488ac6970440570b18d56c044a63b4d630e1f32ad76b04a48f9
```

NAME ↓	TAG	IMAGE ID	CREATED	SIZE
systemdesignschool/kafka-tutor... IN USE	latest	f77194853a2c	1 day ago	1.08 GB

After running this command, a Docker container will be started that contains all the components and code needed for this tutorial. Use the following command to enter the container's interactive environment.

```
docker exec -it kafka-tutorial-practice bash
```

You'll see the following output indicating you've successfully entered the container's interactive environment.

```
~ %>docker exec -it kafka-tutorial-practice bash
root@ca7cc92d6d51:/app#
```

Create a multi-partition topic in Kafka

First, start Kafka by running:

Upon successful startup, you should see the following output:

```
✅ Kafka tutorial environment is ready!
```

Next, let's create a topic with 3 partitions.

```
/opt/kafka/bin/kafka-topics.sh --create --topic multi_partition_ordered --
bootstrap-server localhost:9092 --partitions 3 --replication-factor 1
```

You should see:

```
Created topic multi_partition_ordered.
```

You can check the topic you just created by

```
root@c576023244c2:/app# /opt/kafka/bin/kafka-topics.sh --describe --topic multi_partition_ordered --bootstrap-server localhost:9092 Topic: multi_partition_ordered TopicId: hjyP2c6ETbmqAyIsOWkqig PartitionCount: 3 ReplicationFactor: 1 Configs: min.insync.replicas=1,segment.bytes=1073741824 Topic: multi_partition_ordered Partition: 0 Leader: 1 Replicas: 1 Isr: 1 Elr: LastKnownElr: Topic: multi_partition_ordered Partition: 1 Leader: 1 Replicas: 1 Isr: 1 Elr: LastKnownElr: Topic: multi_partition_ordered Partition: 2 Leader: 1 Replicas: 1 Isr: 1 Elr: LastKnownElr:
```

Create a Kafka producer

Great, now let's create a producer file called `kafka_producer.py`:

```
from kafka import KafkaProducer import time producer = KafkaProducer(bootstrap_servers="localhost:9092") def infinite_number_generator(): num = 0 while True: yield num num += 1 for number in infinite_number_generator(): key = f"key-{number % 3}" # This will ensure we have 3 keys for 3 partitions value = str(number) producer.send( "multi_partition_ordered", key=key.encode("utf-8"), value=value.encode("utf-8") ) time.sleep(1)
```

This producer sends key-value messages with the value being the number starting from 0 to infinity, and keys being `number % 3`. Note that since by default, hash partition by the key is used, each number will be sent to one of the three partitions in a round-robin manner.

This file has already been created in the `/app` directory:

```
root@c576023244c2:/app# ls kafka_consumer.py kafka_producer.py requirements.txt start.sh
```

Run the following command in the `/app` directory to start the producer.

You won't see any output yet. Please keep this terminal open for the next steps. Let's set up the consumer.

Creating Kafka consumers

Now, let's create a consumer file called `kafka_consumer.py`

```
import threading import time from kafka import KafkaConsumer, TopicPartition def consume_partition(partition_id): consumer = KafkaConsumer( bootstrap_servers="localhost:9092", group_id="ordered_partition_group", auto_offset_reset="earliest", ) # Wait for topic to exist and get partitions partitions = [] while not partitions: partitions = list(consumer.partitions_for_topic("multi_partition_ordered")) if not partitions: print(f"Waiting for topic 'multi_partition_ordered' to be created...") time.sleep(1) continue # Check if partition_id is valid if partition_id >= len(partitions): print(f"Partition {partition_id} does not exist. Available partitions: {partitions}") return topic_partition =
```

```
TopicPartition( "multi_partition_ordered", partitions[partition_id] ) consumer.assign([topic_partition]) print(f"Consuming partition {partition_id}") for msg in consumer: print( f"Partition: {msg.partition} - Offset: {msg.offset} - Key: {msg.key} - Value: {msg.value}" ) if __name__ == "__main__": for i in range(3): t = threading.Thread( target=consume_partition, args=(i,), name=f"consume_partition" ) t.start()
```

We create 3 consumers in 3 separate threads, pointing to the same topic. Note that the number of consumers needs to be ≤ the number of partitions because a partition can only be consumed by one consumer at a time in Kafka.

The `kafka_consumer.py` file is also already available in the `/app` directory. Now open a new terminal window, enter the container's interactive environment, and run the consumers

And we see the message being consumed by the consumers.

```
Consuming partition 2 Consuming partition 0 Consuming partition 1 Partition: 1 - Offset: 0 - Key: b'key-0' - Value: b'0' Partition: 1 - Offset: 1 - Key: b'key-0' - Value: b'3' Partition: 1 - Offset: 2 - Key: b'key-0' - Value: b'6' Partition: 1 - Offset: 3 - Key: b'key-0' - Value: b'9' Partition: 1 - Offset: 4 - Key: b'key-0' - Value: b'12' Partition: 1 - Offset: 5 - Key: b'key-0' - Value: b'15' Partition: 1 - Offset: 6 - Key: b'key-0' - Value: b'18' Partition: 1 - Offset: 7 - Key: b'key-0' - Value: b'21' Partition: 1 - Offset: 8 - Key: b'key-0' - Value: b'24' Partition: 1 - Offset: 9 - Key: b'key-0' - Value: b'27' Partition: 1 - Offset: 10 - Key: b'key-0' - Value: b'30' Partition: 1 - Offset: 11 - Key: b'key-0' - Value: b'33' Partition: 1 - Offset: 12 - Key: b'key-0' - Value: b'36' Partition: 1 - Offset: 13 - Key: b'key-0' - Value: b'39' Partition: 1 - Offset: 14 - Key: b'key-0' - Value: b'42' Partition: 1 - Offset: 15 - Key: b'key-0' - Value: b'45' Partition: 1 - Offset: 16 - Key: b'key-0' - Value: b'48' Partition: 1 - Offset: 17 - Key: b'key-0' - Value: b'51' Partition: 1 - Offset: 18 - Key: b'key-0' - Value: b'54' Partition: 1 - Offset: 19 - Key: b'key-0' - Value: b'57' Partition: 1 - Offset: 20 - Key: b'key-0' - Value: b'60' Partition: 1 - Offset: 21 - Key: b'key-0' - Value: b'63' Partition: 1 - Offset: 22 - Key: b'key-0' - Value: b'66' Partition: 1 - Offset: 23 - Key: b'key-0' - Value: b'69'
```

Note that within each partition, the numbers are always monotonically increasing. However, between partitions, the order is not guaranteed as discussed in the last article.